ORGAN DONATION AND PROCUREMENT NETWORK

21CSC205P Database Management Systems

A MINI PROJECT REPORT

Submitted by

Deepthi Chand M (RA2311030010260)

Hemanth (RA2311030010270)

Under the Guidance of

Dr.M.SIVAKUMAR

Assistant Professor, Department of Networking and Communications

in partial fulfillment of the requirements for the degree of

BACHELOR OFTECHNOLOGY

in

COMPUTER SCIENCEAND ENGINEERING

with Specialization
in CYBER SECURITY

DEPARTMENT OFNETWORKINGAND COMMUNICATIONS

COLLEGE OFENGINEERING AND TECHNOLOGY

SRM INSTITUTE OF SCIENCEAND TECHNOLOGY

KATTANKULATHUR– 603 203

MAY 2025

SRM INSTITUTE OF SCIENCEAND TECHNOLOGY

KATTANKULATHUR 603203

BONAFIDE CERTIFICATE

Certified that Project report titled "Organ Donation and procurement Network" is the bonafide work of **"Deepthi Chand M [RA2311030010260], Hemanth [RA2311030010316]"** who carried out the **21CSC205P Database Management Systems** mini project work under my supervision.

COURSE FACULTY

Dr.M.Sivakumar
Assistant Professor
Department of NWC
SRMIST-KTR

HEAD OF THE DEPARTMENT

Dr.M.Lakshmi
Professor and Head
Department of NWC
SRMIST-KTR

ABSTRACT

The Organ Donation and Procurement Network System is a centralized digital platform designed to streamline the complex processes involved in organ donation, allocation, and transplantation. This system facilitates realtime coordination among hospitals, organ banks, transplant centres, and donors, ensuring transparency, traceability, and efficiency in the organ procurement chain. By automating donor registration, medical evaluation, organ matching, and logistics management, the platform minimizes delays and human errors, thereby increasing the success rate of transplants.

Built with a user-friendly interface, the system supports secure data management, prioritization algorithms based on medical urgency and compatibility, and notification services to keep stakeholders informed throughout the donation process. It also ensures regulatory compliance and ethical standards by maintaining accurate records and enabling audits.

Overall, the Organ Donation and Procurement Network System aims to save lives by reducing organ wastage, improving donor-recipient matching, and fostering a culture of timely and ethical organ donation through technological innovation.

Table of Contents

Chapter Number	Description	Page Number
1	Introduction	5
	1.1 Problem understanding, 1.2 Identification of Entity and Relationships, 1.3 Construction of DB using ER Model for the Project	5 5 8
2	Database Design	10
	2.1 Design of Relational Schemas 2.2 Creation of Database Tables for the project	10 11
3	Execution of Queries	14
	3.1 Constraints 3.2 Sets 3.3 Joins 3.4 Trigger 3.5 Cursors 3.6 View	14 15 16 19 22 25
4	Normalization	27
	4.1 Analysing the pitfalls, 4.2 Identifying the dependencies 4.3 Applying normalizations	27 28 29
5	Concurrency control and Recovery mechanisms	35

	5.1 Implementation of Concurrency control 5.1 Implementation of Recovery mechanisms	**35** **36**
6	Code for the project	**37**
7	Result and Discussion (Screenshots of the implementation with front end)	**43**

CHAPTER 1

INTRODUCTION

The current organ donation and procurement process suffers from significant inefficiencies and lacks a unified digital infrastructure to manage donor registration, organ matching, logistics coordination, and real-time updates among hospitals, organ banks, and recipients. This fragmented system leads to delays in transplantation, miscommunication between stakeholders, loss of viable organs, and reduced trust in the process. Therefore, a centralized, secure, and user-friendly system is essential to streamline operations, enhance transparency, and improve the success rate of organ transplants.

.

1.1 Problem Statement:

1. **Manual and Paper-Based Processes:** Donor registration, organ availability tracking, and recipient matching are often performed manually, resulting in delayed processing and higher chances of error.

2. **Lack of Real-Time Coordination:** Hospitals and organ banks operate in silos, leading to slow communication and coordination, which can critically affect organ viability and transplant outcomes.

3. **Inconsistent Data Management:** Absence of a centralized system causes discrepancies in donor and recipient data, complicating organ matching and leading to potential mismatches.

4. **Limited Transparency and Traceability:** Stakeholders have minimal visibility into the organ procurement and allocation process, leading to mistrust and potential ethical concerns.

5. **Inefficient Logistics and Transportation Tracking:** Organ transport logistics are not always properly coordinated or tracked in real time, resulting in delays and potential wastage of organs.

6. **Security and Privacy Risks:** Sensitive personal and medical information of donors and recipients is vulnerable in unprotected systems, posing risks to data privacy and compliance with regulations.

7. **Poor User Experience:** Existing systems (if any) may not offer intuitive interfaces for different users (e.g., doctors, coordinators, donors), making it difficult to efficiently navigate and perform critical tasks.

1.2 ER Analysis: Identifying Entity Sets and Relationship Sets:

1.2.1 User

The User entity represents any individual who interacts with the organ donation system. This includes donors and patients. It stores general personal and medical information necessary to identify and classify the users.

Attributes:

- User ID: A unique identifier for each user.
- Name: Full name of the user.
- Date of Birth: User's date of birth for verifying age.
- Phone Number (multi-valued): Users may have more than one contact number.
- Medical Insurance: Details of the user's medical insurance coverage.
- Medical History: Previous health conditions, surgeries, or treatments.
- Address: Residential address for communication and verification.

1.2.2 Patient

The Patient entity stores information about users who require an organ transplant. Each patient is linked to a User entity and includes specific transplant-related data.

Attributes:

- Patient ID: Unique identification for each patient.
- Organ Required: The specific organ that the patient needs.
- Reason of Procurement: Medical reason or diagnosis for needing the organ.
- User ID (foreign key): Refers to the related record in the User entity.

1.2.3 Donor

The Donor entity holds details about individuals who have registered to donate one or more organs. It may include both living and deceased donors.

Attributes:

- Donor ID: Unique identification for each donor.
- Organ Donated: The organ or organs being donated by the individual.
- Reason of Donation: The reason or motivation behind the donation.
- User ID (foreign key): Links this donor record to the corresponding User.

1.2.4 Organ Available

This entity represents organs that are currently available for transplantation in the system. It connects each available organ to its donor.

Attributes:

- Organ ID: Unique identifier for each organ record.
- Organ Name: Type of organ available (e.g., Heart, Liver).
- Donor ID (foreign key): Refers to the donor who provided the organ.

1.2.5 Organization

The Organization entity stores data about institutions such as hospitals and NGOs involved in organ donation and transplantation. These organizations must be government-approved to operate.

Attributes:

- Organization ID: Unique identifier for the organization.
- Organization Name: Official name of the institution.
- Location: Geographic location of the organization.
- Government Approved Organization or Not: Indicates government approval status.
- Phone Number (multi-valued): Multiple contact numbers for the organization.

1.2.6 Doctor

The Doctor entity stores details about the medical professionals who are involved in performing, approving, or managing transplants within the system.

Attributes:

- Doctor ID: Unique identification for each doctor.
- Doctor Name: Full name of the doctor.
- Phone Number (multi-valued): One or more phone numbers for contacting the doctor.
-

1.2.7 Organization Head

This entity represents the individual who serves as the head of a registered organization. It stores administrative and tenure details of the organizational leader. Attributes:

- Head Name: Full name of the head of the organization.
- Date of Joining: The date the head assumed their position.
- Term Length: Duration of the head's service period. **Relationship Sets**

Donor – Organ

Type: Many-to-Many

Explanation: A donor can donate multiple types of organs, and an organ type can be donated by multiple donors.

Patient – Organ

Type: Many-to-One

Explanation: A patient requires a specific type of organ, but one organ type may be needed by many patients.

Doctor – Organization Type: Many-to-One

Explanation: A doctor is associated with one organization, but an organization can have multiple doctors.

Transaction – Donor Type:

Many-to-One

Explanation: Each transaction is linked to one donor, but a donor can be involved in multiple transactions.

Transaction – Patient

Type: Many-to-One

Explanation: Each transaction involves one patient, though a patient may have multiple transactions (e.g., failed or successful attempts). Transaction – Organ Type: Many-to-One

Explanation: Each transaction is linked to one organ, but an organ type may appear in many transactions.

Transaction – Doctor Type:

Many-to-One

Explanation: A doctor performs many transplant procedures, but each transaction involves one doctor.

Transaction – Organization

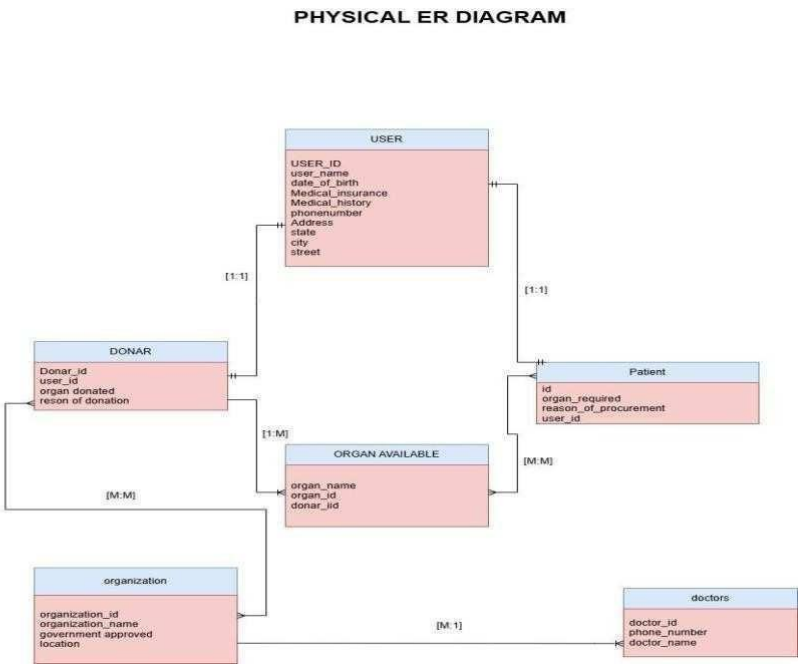
Type:Many-toOne

Explanation: Each transaction is conducted under one organization, but the same organization can handle many transactions.

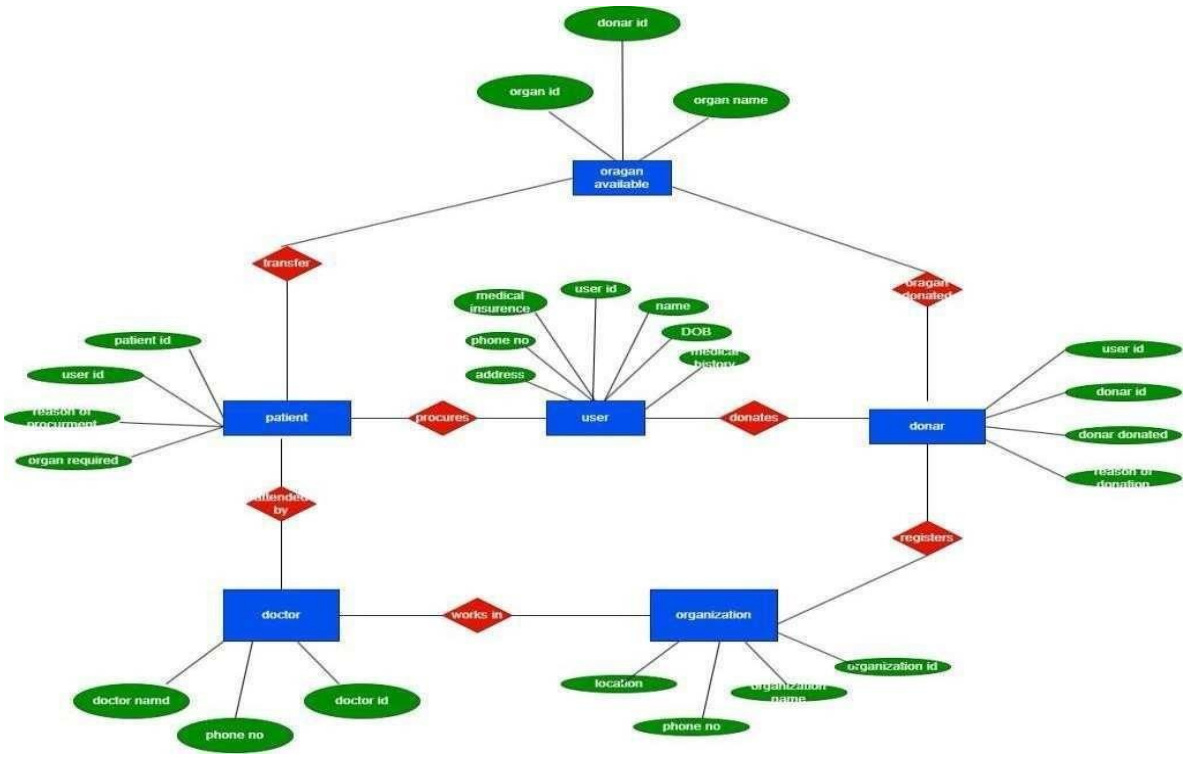
1.3 Construction of DB using ER Model for the project

This ER diagram for the Organ Donation System captures a robust and traceable approach to tracking both donors and recipients. It accounts for all medical, administrative, and ethical considerations by connecting users, patients, donors, medical staff, and organizational authorities. Relationships such as DONATES, PROCURES, and ATTENDED BY ensure the logical flow of responsibilities and data. This system structure promotes efficient management and transparency in life-saving organ transplants.

Physical Diagram :



Logical Diagram:



CHAPTER 2 DATABASE DESIGN

2.1. Design of Relational Schemas

The relational schema represents the structure of your database, defining the tables and relationships between them.

Entities and Their Relationships:

2.1.1. User (stores information about users, both doctors and patients)

- user_id (Primary Key)
- name
- email
- phone
- role (Doctor, Patient, etc.) - password (hashed for security)

2.1.2. Organization (stores hospital or donation organization details)

- organization_id (Primary Key)
- name
- address
- contact_number

2.1.3. Doctor (doctors who handle organ reservations)

- doctor_id (Primary Key)
- user_id (Foreign Key referencing User)
- organization_id (Foreign Key referencing Organization) - specialization

2.1.4. Donor (donors who are donating organs) -

donor_id (Primary Key)

- user_id (Foreign Key referencing User)
- blood_group
- contact_number
- status (e.g., Available, Not Available)

2.1.5. Organ_Available (tracks available organs for transplantation)

- organ_id (Primary Key)
- donor_id (Foreign Key referencing Donor)
- organ_type (e.g., Kidney, Heart)
- status (e.g., Available, Reserved)
- date_of_availability

2.1.6. Patient (patients waiting for organ transplantation)

- patient_id (Primary Key)
- user_id (Foreign Key referencing User)
- required_organ (e.g., Kidney, Heart)
- blood_group

2.1.7. Transaction (tracks organ transactions between donors and patients)

- transaction_id (Primary Key)
- organ_id (Foreign Key referencing Organ_Available) - patient_id (Foreign Key referencing Patient)
- transaction_date
- status (e.g., Pending, Completed)
-

2.2. Creation of Database Tables

Below are the SQL CREATE TABLE statements based on the relational schema:

```
-- Creating the 'user' table
CREATE TABLE user ( user_id INT AUTO_INCREMENT
PRIMARY KEY, name VARCHAR(255), email VARCHAR(255)
UNIQUE, phone VARCHAR(15), role ENUM('Doctor', 'Patient',
'Admin') NOT NULL, password VARCHAR(255) NOT NULL
);

-- Creating the 'organization' table CREATE
TABLE organization (
```

```
organization_id INT AUTO_INCREMENT PRIMARY KEY,name
VARCHAR(255), address TEXT, contact_number VARCHAR(15)
);

-- Creating the 'doctor' table
CREATE TABLE doctor ( doctor_id INT AUTO_INCREMENT
    PRIMARY KEY user_id INT,
organization_id INT, specialization VARCHAR(100),
FOREIGN KEY (user_id) REFERENCES user(user_id),
FOREIGN KEY (organization_id) REFERENCES organization(organization_id)
);

-- Creating the 'donor' table
CREATE TABLE donor ( donor_id INT AUTO_INCREMENT
    PRIMARY KEY, user_id INT, blood_group VARCHAR(10),
    contact_number VARCHAR(15), status ENUM('Available', 'Not
Available') NOT NULL, FOREIGN KEY (user_id) REFERENCES
user(user_id)
);

-- Creating the 'organ_available' table
CREATE TABLE organ_available ( organ_id INT
AUTO_INCREMENT PRIMARY KEY, donor_id INT, organ_type VARCHAR(50),
status ENUM('Available', 'Reserved') NOT
NULL,
date_of_availability DATE,
FOREIGN KEY (donor_id) REFERENCES donor(donor_id)
);

-- Creating the 'patient' table
CREATE TABLE patient (
patient_id INT AUTO_INCREMENT PRIMARY KEY, user_id INT,
    required_organ VARCHAR(50), blood_group
VARCHAR(10), FOREIGN KEY (user_id) REFERENCES
user(user_id)
);

-- Creating the 'transaction' table
CREATE TABLE transaction ( transaction_id INT AUTO_INCREMENT
PRIMARY KEY, organ_id INT, patient_id INT, transaction_date
DATETIME,
status ENUM('Pending', 'Completed') NOT NULL,
```

```
FOREIGN KEY (organ_id) REFERENCES organ_available(organ_id),
FOREIGN KEY (patient_id) REFERENCES patient(patient_id) );
```

2.3 . Sample Data Insertion

Below are SQL commands for inserting sample data into the tables:

```
-- Inserting sample data into 'user' table
INSERT INTO user (name, email, phone, role, password)
VALUES ('Dr. John Doe', 'johndoe@example.com', '9876543210', 'Doctor',
'hashed_password'),
('Alice Smith', 'alice.smith@example.com', '9876543211', 'Patient', 'hashed_password'),
('Dr. Jane Doe', 'janedoe@example.com', '9876543212', 'Doctor', 'hashed_password'),
('Bob Brown', 'bob.brown@example.com', '9876543213', 'Patient', 'hashed_password'),
('Dr. Emily White', 'emily.white@example.com', '9876543214', 'Doctor', 'hashed_password'),
('Carol Green', 'carol.green@example.com', '9876543215', 'Patient', 'hashed_password'),
('Dr. Michael Black', 'michael.black@example.com', '9876543216', 'Doctor', 'hashed_password'),
('David Gray', 'david.gray@example.com', '9876543217', 'Patient', 'hashed_password'),
('Dr. Sarah Blue', 'sarah.blue@example.com', '9876543218', 'Doctor', 'hashed_password'),
('Eva Pink', 'eva.pink@example.com', '9876543219', 'Patient', 'hashed_password'),
('Dr. Jack Orange', 'jack.orange@example.com', '9876543220', 'Doctor', 'hashed_password'),
('Nina Violet', 'nina.violet@example.com', '9876543221', 'Patient', 'hashed_password'),
('Dr. William Red', 'william.red@example.com', '9876543222', 'Doctor', 'hashed_password'),
('Paul Yellow', 'paul.yellow@example.com', '9876543223', 'Patient', 'hashed_password'),
('Dr. Olivia Gray', 'olivia.gray@example.com', '9876543224', 'Doctor', 'hashed_password');

-- Inserting sample data into 'organization' table
INSERT INTO organization (name, address, contact_number)
VALUES ('City Hospital', '123 Main St, Cityville', '1234567890'),
('Global Medical Center', '456 Broad Ave, Metropolis', '1234567891'),
('St. John's Hospital', '789 Park Rd, Smalltown', '1234567892'),
('Mountain Health', '101 Mountain Rd, Hilltop', '1234567893'),
('Bright Life Clinic', '202 Greenfield St, Greentown', '1234567894');

-- Inserting sample data into 'doctor' table
INSERT INTO doctor (user_id, organization_id, specialization)
VALUES (1, 1, 'Nephrologist'),
(3, 2, 'Cardiologist'),
(5, 3, 'Orthopedic'),
(7, 4, 'Neurologist'),
(9, 5, 'Pediatrician');
```

```
-- Inserting sample data into 'donor' table
INSERT INTO donor (user_id, blood_group, contact_number, status)
VALUES (1, 'O+', '9876543210', 'Available'),
(3, 'A-', '9876543211', 'Available'),
(5, 'B+', '9876543212', 'Not Available'),
(7, 'AB+', '9876543213', 'Available'),
(9, 'O-', '9876543214', 'Available');

-- Inserting sample data into 'organ_available' table
INSERT INTO organ_available (donor_id, organ_type, status, date_of_availability)
VALUES (1, 'Kidney', 'Available', '2025-05-01'),
(3, 'Heart', 'Available', '2025-06-01'),
(5, 'Liver', 'Not Available', '2025-07-01'),
(7, 'Kidney', 'Reserved', '2025-08-01'),
(9, 'Lung', 'Available', '2025-09-01');

-- Inserting sample data into 'patient' table
INSERT INTO patient (user_id, required_organ, blood_group)
VALUES (2, 'Kidney', 'O+'),
(4, 'Heart', 'A-'),
(6, 'Liver', 'B+'),
(8, 'Kidney', 'AB+'),
(10, 'Lung', 'O-');

-- Inserting sample data into 'transaction' table
INSERT INTO transaction (organ_id, patient_id, transaction_date, status)
VALUES (1, 1, NOW(), 'Pending'),
(2, 2, NOW(), 'Pending'),
 (4, 3, NOW(), 'Completed'),(5, 4, NOW(),
'Pending'),(7, 5, NOW(), 'Completed');
```

CHAPTER 3 QUERY EXECUTION

3.1 Constraints

Not Null – Ensures a column cannot have a NULL value.

Primary Key – Uniquely identifies each row; combines NOT NULL + UNIQUE.
Foreign Key – Ensures values in one table match values in another (referential integrity).

Check – Ensures values meet a specific condition.

Default – Assigns a default value if no value is provided.

3.1.1. Not Null

```
CREATE TABLE user ( user_id INT
PRIMARY KEY, name VARCHAR(100)
NOT NULL, age INT, gender
VARCHAR(10)
);
```

3.1.2.Pimary key

```
CREATE TABLE doctor (
doctor_id INT PRIMARY KEY,
name VARCHAR(100) NOT NULL
);
```

3.1.3. Foreign Key

Link the doctor_id in the organ_available table to the doctor table.

```
CREATE TABLE organ_available ( organ_id INT PRIMARY KEY, organ_name VARCHAR(50), doctor_id INT,
FOREIGN KEY (doctor_id) REFERENCES doctor(doctor_id)
);X`
```

3.1.4.Check

```
CREATE TABLE patient ( patient_id
INT PRIMARY
KEY, name
VARCHAR(100), age INT
CHECK (age > 0)
);
```

3.1.5 Unique

```
ALTER TABLE user
ADD CONSTRAINT unique_email UNIQUE (email),
ADD CONSTRAINT unique_phone UNIQUE (phone);
```

3.2SetOperation
Query:

```
SELECT U.Name FROM User U
```

JOIN Patient P ON U.UserID = P.UserID UNION SELECT U.Name FROM User U

JOIN Donor D ON U.UserID = D.UserID;

RESULT TABLE

Name
Alice Johnson
Bob Smith
Charlie Brown
David Wilson
Eve Davis

UNION removes duplicates; every user who is either a Patient or Donor.

1. Join Query (Patient–Doctor–User) Setup (intermediate table):

CREATE TABLE PatientDoctor
(PatientID INT,
DoctorID INT,
FOREIGN KEY (PatientID)

REFERENCES Patient(PatientID),
FOREIGN KEY (DoctorID) REFERENCES
Doctor(DoctorID)
);

Sample Data:

INSERTINTO PatientDoctorVALUES

(1, 1), -- Alice → Dr. Green

(2, 2), -- Bob → Dr. Black

(3, 3), -- Charlie → Dr. White (4, 4), -- David
→ Dr. Blue (5, 5); -- Eve → Dr. Yellow

3.3 Join Query:

SELECT

U.Name AS PatientName,

P.OrganRequired,

D.DoctorName,

D.Department

FROM Patient P

JOIN User U ON P.UserID = U.UserID

JOIN PatientDoctor PD ON P.PatientID =

PD.PatientID;

JOIN DoctorD ON PD.DoctorID=

D.DoctorID;

PatientName	OrganRequired	DoctorName	Department
Alice Johnson	Kidney	Dr. Green	Nephrology
Bob Smith	Liver	Dr. Black	Hepatology
Charlie Brown	Heart	Dr. White	Cardiology
David Wilson	Lung	Dr. Blue	Pulmonology
Eve Davis	Pancreas	Dr. Yellow	Endocrinology

A.Union All mysql> SELECT

Doctor_name FROM doctor

-> UNION ALL

-> SELECT Organization_name FROM organization;

+----------+

| Doctor_name |

+----------+

| Dr. Alice White |

| Dr. Bob Green | | Dr. Charlie Blue |

| Dr. Daisy Black |

| Dr. Edward Grey |

| Dr. Fiona Purple |

| Dr. George Red |

| Dr. Helen Yellow |

| Dr. Isaac Brown |

| Dr. Jackie Pink |
| Dr. Kevin Orange |
| Dr. Linda Blue |
| Dr. Michael Green |
| Dr. Nora Black |
| Dr. Oscar White |
| Care Foundation |
| Care Health |
| Gift of Life |
| Global Health |
| Health Support |
| HealthCare Org |
| Hope Charity |
| Hope Foundation |
| Hope Life |
| Life Foundation |
| LifeSaver Org |
| Medical Assistance |
| Safe Health |
| Wellness Aid |
| Wellness Center |
+----------+

B. Union

SELECT Doctor_name FROM doctor
UNION
SELECT Organization_name FROM organization; OUTPUT:
+----------+
| Doctor_name | +----------+
| Dr. Alice White|
| Dr. Bob Green | |
Dr. Charlie Blue |
| Dr. Daisy Black| |
Dr. Edward Grey|
| Dr. Fiona Purple |
| Dr. George Red |
| Dr. Helen Yellow |

| Dr. Isaac Brown|

| Dr. Jackie Pink|

| Dr. Kevin Orange |

| Dr. Linda Blue |

| Dr. Michael Green |

| Dr. Nora Black |

| Dr. Oscar White| |

Care Foundation|

| Care Health |

| Gift of Life |

| Global Health | |

Health Support |

| HealthCare Org |

| Hope Charity | |

Hope Foundation|

| Hope Life | |

Life Foundation|

| LifeSaver Org |

| Medical Assistance |

| Safe Health |

| Wellness Aid |

| Wellness Center|

+----------+

30 rows in set (0.00 sec

mysql> SELECT d.Doctor_name, o.Organization_name

-> FROM doctor d

-> INNER JOIN organization o ON d.Organization_ID = o.Organization_ID;

+----------+----------+

| Doctor_name | Organization_name |

+----------+----------+ |

Dr. Alice White| HealthCare Org |

| Dr. Bob Green | Hope Foundation | | Dr. Charlie

Blue | Care Health |

| Dr. Daisy Black| Wellness Center |

| Dr. Edward Grey| Gift of Life |

| Dr. Fiona Purple | LifeSaver Org |

| Dr. George Red| Medical Assistance |

| Dr. Helen Yellow | Hope Charity | |

Dr. Isaac Brown | Health Support

|

| Dr. Jackie Pink| Care Foundation |

| Dr. Kevin Orange | Life Foundation |

| Dr. Linda Blue| Safe Health |

| Dr. Michael Green | Global Health |

| Dr. Nora Black| Hope Life |

| Dr. Oscar White| Wellness Aid |

+----------+----------+

15 rows in set (0.00 sec) mysql>

SELECT * FROM patient

-> WHERE Patient_ID NOT IN (SELECT Patient_ID FROM transaction WHERE Status = 'Completed');

+----------+----------+----------+----------+----------+

| Patient_ID | Organ_req | Reason_of_procurement| Doctor_ID | User_ID |

+----------+----------+----------+----------+----------+

| 3 | Heart | Heart failure | 3 3 |

|

| 5 | | Vision loss | 5 5 |

Cornea |

| 7 | Heart | Cardiac arrest | 7 7 |

|

| 8 | Liver | Liver cancer | 8 8 |

|

| 10 | | Diabetic nephropathy | 10 | 10

Kidney |

| 12 | Heart | Coronary artery disease | 12 | 12 |

| 13 | Cornea | Corneal dystrophy | 13 | 13 |

| 15 | Liver | Fatty liver disease | 15 | 15 |

+----------+----------+----------+----------+----------+

8 rows in set (0.01 sec)

3.4 Trigger

1.Before Delete

mysql> DELIMITER // mysql> CREATE TRIGGER before_delete_patient

-> BEFORE DELETE ON patient

-> FOR EACH ROW

-> BEGIN

-> -- Example: Prevent deletion if patient is linked to a transaction

-> IF EXISTS (SELECT 1 FROM transaction WHERE Patient_ID = OLD.Patient_ID) THEN

-> SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Patient cannot be deleted because they are involved in a transaction';

-> END IF;

-> END;

-> //

Query OK, 0 rows affected (0.07 sec)

DELETE FROM patient WHERE Patient_ID = 101;

Result:

ERROR 1644 (45000): Patient cannot be deleted because they are involved in a transaction

2.After Delete

mysql> DELIMITER ; mysql> DELIMITER // mysql>

CREATE TRIGGER after_delete_donor

-> AFTER DELETE ON donor

-> FOR EACH ROW

-> BEGIN

-> -- Example: Clean up associated organ availability

-> DELETE FROM organ_available WHERE Donor_ID = OLD.Donor_ID;

-> END;

-> //

Query OK, 0 rows affected (0.02 sec) DELETE FROM donor
WHERE Donor_ID = 202;

Result:

Query OK, 1 row affected

-- (Organs associated with Donor_ID = 202 also deleted)

3: After Insert

mysql> DELIMITER ; mysql> DELIMITER // mysql> CREATE TRIGGER after_insert_transaction

-> AFTER INSERT ON transaction

-> FOR EACH ROW

-> BEGIN

-> -- Example: Log insertion of a new transaction

-> INSERT INTO transaction_log (Transaction_ID, Action)

-> VALUES (NEW.Patient_ID, 'Transaction Completed'); > END;

-> //

Query OK, 0 rows affected (0.02 sec)

INSERT INTO transaction (Transaction_ID, Patient_ID, Donor_ID, Date) VALUES (301, 101, 202, '2025-04-20'); Result:

Query OK, 1 row affected

-- Log entry added to transaction_log: (101, 'Transaction Completed')

4: Before Insert

```
mysql> DELIMITER ; mysql> DELIMITER // mysql> CREATE
TRIGGER before_insert_patient
-> BEFORE INSERT ON patient
-> FOR EACH ROW
-> BEGIN
-> -- Example: Ensure the patient has valid doctor assignment
-> IF NEW.Doctor_ID IS NULL THEN
->    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Doctor assignment is mandatory';
-> END IF;
-> END;
-> //
Query OK, 0 rows affected (0.03 sec)
INSERT INTO patient (Patient_ID, Name, Age, Organ_req, Doctor_ID) VALUES (401,
'John Doe', 45, 'Liver', NULL); Result:
ERROR 1644 (45000): Doctor assignment is mandatory
```

5: After Update

```
mysql> DELIMITER ; mysql> DELIMITER // mysql> CREATE TRIGGER
after_update_organ_status
-> AFTER UPDATE ON organ_available
-> FOR EACH ROW
-> BEGIN
-> -- Example: Update organ status after change in availability
-> IF NEW.Donor_ID IS NULL THEN
-> UPDATE organ_available SET Status = 'Unavailable' WHERE Organ_ID = NEW.Organ_ID;
-> END IF;
-> END;
-> //
Query OK, 0 rows affected (0.04 sec)

UPDATE organ_available SET Donor_ID = NULL WHERE Organ_ID = 501; Result:
Query OK, 1 row affected
-- Status of Organ_ID = 501 changed to 'Unavailable'
```

6: Before_Update mysql>

DELIMITER ;

mysql> DELIMITER // mysql> CREATE TRIGGER before_update_donor_status

-> BEFORE UPDATE ON donor

-> FOR EACH ROW

-> BEGIN

-> -- Example: Prevent change if donor status is invalid

-> IF NEW.Reason_of_donation IS NULL THEN

-> SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Reason for donation cannot be null';

-> END IF;

-> END;

-> //

Query OK, 0 rows affected (0.03 sec)

UPDATE donor SET Reason_of_donation = NULL WHERE Donor_ID = 202;

Result:

ERROR 1644 (45000): Reason for donation cannot be null

3.5 Cursor

mysql>DELIMITER$$ mysql>CREATE PROCEDURE cursor_example()

```
-> BEGIN
->    DECLARE done INT DEFAULT 0;
->    DECLARE patient_id INT;
->    DECLARE organ_req VARCHAR(50);
->    DECLARE cur CURSOR FOR
->        SELECT Patient_ID, Organ_req FROM patient;
->    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
->    OPEN cur;
->    read_loop: LOOP
->        FETCH cur INTO patient_id, organ_req;
->        IF done THEN
->            LEAVE read_loop;
->        END IF;
->        -- Example: Print patient_id and organ_req (use any logic here)
->        SELECT patient_id, organ_req;
->    END LOOP;
```

```
->    CLOSE cur;
-> END$$
Query OK, 0 rows affected (0.01 sec) mysql>
        DELIMITER ;           mysql>
CALL cursor_example();
+------------+-----------+
| patient_id | organ_req | +------------+-----------+
 | NULL | NULL      |
+------------+-----------+
1 row in set (0.00 sec)

+------------+-----------+
| patient_id | organ_req | +------------+-----------+
 | NULL | NULL      |
+------------+-----------+
1 row in set (0.01 sec)

+------------+-----------+
| patient_id | organ_req | +------------+-----------+
 | NULL | NULL      |
+------------+-----------+
1 row in set (0.01 sec)

+------------+-----------+
| patient_id | organ_req |
+------------+-----------+
 | NULL | NULL      |
+------------+-----------+
1 row in set (0.01 sec)

+------------+-----------+
| patient_id | organ_req |
+------------+-----------+
 | NULL | NULL      |
+------------+-----------+
1 row in set (0.01 sec)

+------------+-----------+
| patient_id | organ_req |
+------------+-----------+
 | NULL | NULL      |
+------------+-----------+
```

1 row in set (0.01 sec)

```
+------------+-----------+
| patient_id | organ_req |
+------------+-----------+
 | NULL | NULL      |
+------------+-----------+
```

1 row in set (0.02 sec)

```
+------------+-----------+
| patient_id | organ_req |
+------------+-----------+
 | NULL | NULL      |
+------------+-----------+
```

1 row in set (0.02 sec)

```
+------------+-----------+
| patient_id | organ_req |
+------------+-----------+
 | NULL | NULL      |
+------------+-----------+
```

1 row in set (0.02 sec)

```
+------------+-----------+
| patient_id | organ_req |
+------------+-----------+
 | NULL | NULL      |
+------------+-----------+
```

1 row in set (0.02 sec) +------------+-----------+

```
| patient_id | organ_req |
+------------+-----------+
 | NULL | NULL      |
+------------+-----------+
```

1 row in set (0.03 sec)

```
+------------+-----------+
| patient_id | organ_req |
+------------+-----------+
 | NULL | NULL      |
+------------+-----------+
```

1 row in set (0.03 sec)

```
+------------+-----------+
```

| patient_id | organ_req |
+----------+----------+
| NULL | NULL |
+----------+----------+
1 row in set (0.03 sec)

+----------+----------+
| patient_id | organ_req |
+----------+----------+
| NULL | NULL |
+----------+----------+
1 row in set (0.03 sec)

+----------+----------+
| patient_id | organ_req |
+----------+----------+
| NULL | NULL |
+----------+----------+
1 row in set (0.03 sec)

3.6 View

3.6.1. Inner Join View:

mysql> CREATE VIEW patient_organ_info AS -> SELECT p.Patient_ID, p.Name, p.Organ_req, o.Organ_type

-> FROM patients p

-> INNER JOIN organs o

-> ON p.Patient_ID = o.Patient_ID; Query OK, 0 rows

affected (0.01 sec) Output: mysql> SELECT * FROM

patient_organ_info;

+----------+----------+----------+----------+
| Patient_ID | Name | Organ_req | Organ_type |
+----------+----------+----------+----------+
1	John Doe	Heart	Heart
2	Jane Smith	Kidney	Kidney
3	Alice Johnson	Liver	Liver
4	Bob Brown	Lung	Lung
+----------+----------+----------+----------+
4 rows in set (0.00 sec) 3.4.2.

3.6.2 Left Join View:

mysql> CREATE VIEW patient_organ_left_info AS ->
SELECT p.Patient_ID, p.Name, p.Organ_req, o.Organ_type
-> FROM patients p
-> LEFT JOIN organs o
-> ON p.Patient_ID = o.Patient_ID; Query OK,
0 rows affected (0.01 sec)

Output:
mysql> SELECT * FROM patient_organ_left_info;
+----------+----------+----------+----------+
| Patient_ID | Name | Organ_req | Organ_type |
+----------+----------+----------+----------+
1	John Doe	Heart	Heart
2	Jane Smith	Kidney	Kidney
3	Alice Johnson	Liver	Liver
4	Bob Brown	Lung	Lung
+----------+----------+----------+----------+ 4
rows in set (0.00 secc

3.6.3.Right Join View:

mysql> CREATE VIEW organ_patient_right_info AS -> SELECT p.Patient_ID, p.Name, p.Organ_req, o.Organ_type -> FROM patients p
-> RIGHT JOIN organs o
-> ON p.Patient_ID = o.Patient_ID; Query
OK, 0 rows affected (0.02 sec) Output:
mysql> SELECT * FROM organ_patient_right_info;
+----------+----------+----------+----------+
| Patient_ID | Name | Organ_req | Organ_type |
+----------+----------+----------+----------+
1	John Doe	Heart	Heart
2	Jane Smith	Kidney	Kidney
3	Alice Johnson	Liver	Liver
4	Bob Brown	Lung	Lung

4 rows in set (0.00 sec)

CHAPTER 4 NORMALIZATION

4.1Analysing Pitfalls

In the development of the Secure User Authentication System, several critical pitfalls can affect data integrity, performance, and system security. These challenges often stem from poor database normalization, redundant data storage, and the absence of proper constraints. Below are key issues identified and corresponding solutions in the context of this authentication system.

a) Data Redundancy Issue:

Data redundancy occurs when the same information—such as user credentials or contact details—is stored in multiple places unnecessarily. For instance, storing username or email in both the users and login_sessions tables leads to potential inconsistencies when user details are updated.

Impact:

- Inconsistent user data across tables
- Higher storage costs
- Complexity in maintaining updates Solution:

Normalize the database design by storing each attribute in one location only and referencing it through foreign keys. The login_sessions table should reference user_id from the users table, not store redundant user information.

Example: Redundant Username in Sessions Table CREATE

TABLE login_sessions_with_redundancy

```
( session_id INT AUTO_INCREMENT PRIMARY KEY, user_id INT
NOT NULL, username VARCHAR(50), --
Redundant field
login_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (user_id) REFERENCES users(user_id)
);

INSERT INTO login_sessions_with_redundancy (user_id, username, login_time)
VALUES
(101, 'john_doe', NOW()), (102,
'alice_smith', NOW());
```

b) Lack of Proper Constraints Issue:

Without foreign key constraints, invalid or orphaned data can enter the system—for example, creating a login session for a non-existent user.

28

Problem Scenario:

```
INSERT INTO login_sessions (user_id, login_time)
```

VALUES (999, NOW()); -- Assuming user_id 999 does not exist Expected

Result:

The insertion should fail if a foreign key constraint is defined on user_id.

Solution:

Always define FOREIGN KEY constraints between session-related tables and the main users table to enforce referential integrity.

4.2 Identifying the Dependencies

Understanding attribute dependencies helps maintain consistency and ensures efficient database normalization in a secure authentication system.

1. Functional Dependency

One attribute uniquely determines another.

Example:user_id→email

,user_id→hashed_password

SELECT user_id, email FROM users WHERE user_id = 1;

2. Transitive Dependency

An attribute depends on another through an intermediate

attribute. Example: user_id→session_id→login_time

If login_time depends on session_id which depends on user_id, it's a transitive dependency.

SELECT session_id, login_time FROM login_sessions WHERE user_id = 1;

3. Multivalued Dependency (MVD)

An attribute has multiple independent values for another.

Example:

A user may have multiple recovery questions and multiple associated answers.

SELECT user_id, question_text FROM recovery_questions WHERE user_id = 1;

4. Join Dependency (JD)

A table can be decomposed into multiple tables and reconstructed through a join.

Example: users, recovery_questions, and user_logs tables may be joined to form a complete view of a user's security profile.

```
SELECT u.user_id, u.email, r.question_text, l.login_time
FROM users u
JOIN recovery_questions r ON u.user_id = r.user_id
```

JOIN login_sessions l ON u.user_id = l.user_id;

4.3 Applying Normalization

Note:

The normalization part of this project uses a student records system to explain normalization from 1NF to 5NF. It includes student IDs, names, courses (like DBMS, Java, AI), and grades. This example was used because the Secure User Authentication System is already normalized, so it's not useful for showing each step. The student data makes it easier to understand different types of dependencies in a clear way.

- **Zero Normal Form (0NF):**

Zero Normal Form (0NF) refers to the unnormalized state of a database table. In this form, the table may contain repeating groups, multi-valued attributes, or composite fields, and there is no adherence to relational database principles. This is the raw form of data before any normalization is applied.

```
mysql> select*from StudentCourse_0NF;
+-----------+--------+----------------------+--------+
| StudentID | Name   | Courses              | Grades |
+-----------+--------+----------------------+--------+
|       201 | Zara   | WebDev,Cloud         | A,B    |
|       202 | Liam   | DataViz,CyberSec,IoT | A,C,B  |
|       203 | Nina   | DevOps               | A      |
|       204 | Ryan   | Cloud,DataViz        | B,A    |
|       205 | Sophia | IoT,WebDev,CyberSec  | C,B,A  |
+-----------+--------+----------------------+--------+
5 rows in set (0.00 sec)
```

- **First Normal Form (1NF):**

Ensure that each column contains only atomic values and that each record is unique. In an online food ordering system, this means ensuring that fields like **menu_item_name** and **order_date** do not store multiple values (e.g., a list of menu items in a single field). Each order and menu item must be stored as separate records.

- **Second Normal Form (2NF):**

Eliminate partial dependencies. If an attribute depends on part of the primary key, it violates 2NF. In the case of a composite primary key, we ensure that every non-key attribute depends on the entire key. For example, in the Order_Items table, attributes like quantity and price must depend on both order_id and menu_item_id (the composite key), not just one of them.

```
mysql> select * from StudentCourse_1NF;
+-----------+--------+----------+-------+
| StudentID | Name   | Course   | Grade |
+-----------+--------+----------+-------+
|       201 | Zara   | WebDev   | A     |
|       201 | Zara   | Cloud    | B     |
|       202 | Liam   | DataViz  | A     |
|       202 | Liam   | CyberSec | C     |
|       202 | Liam   | IoT      | B     |
|       203 | Nina   | DevOps   | A     |
|       204 | Ryan   | Cloud    | B     |
|       204 | Ryan   | DataViz  | A     |
|       205 | Sophia | IoT      | C     |
|       205 | Sophia | WebDev   | B     |
|       205 | Sophia | CyberSec | A     |
+-----------+--------+----------+-------+
11 rows in set (0.00 sec)
```

```
mysql> select * from  Student_2NF;
+-----------+--------+
| StudentID | Name   |
+-----------+--------+
|       201 | Zara   |
|       202 | Liam   |
|       203 | Nina   |
|       204 | Ryan   |
|       205 | Sophia |
+-----------+--------+
5 rows in set (0.00 sec)

mysql> select * from Enrollment_2NF;
+-----------+----------+-------+---------------+
| StudentID | Course   | Grade | Department    |
+-----------+----------+-------+---------------+
|       201 | Cloud    | B     | Cloud Tech    |
|       201 | WebDev   | A     | Software Eng  |
|       202 | CyberSec | C     | Cybersecurity |
|       202 | DataViz  | A     | Software Eng  |
|       202 | IoT      | B     | Cloud Tech    |
|       203 | DevOps   | A     | Software Eng  |
|       204 | Cloud    | B     | Cloud Tech    |
|       204 | DataViz  | A     | Software Eng  |
|       205 | CyberSec | A     | Cybersecurity |
|       205 | IoT      | C     | Cloud Tech    |
|       205 | WebDev   | B     | Software Eng  |
+-----------+----------+-------+---------------+
11 rows in set (0.00 sec)
```

- **Third Normal Form (3NF):**

Eliminate transitive dependencies. This involves ensuring that every non-key attribute is directly dependent on the primary key, and not on other non-key attributes. For instance, in the Orders table, the canteen_name should not be stored directly since it depends on canteen_id.

Instead, canteen_name should be retrieved from the Canteens table using canteen_id.

```
mysql> select * from Enrollment_2NF;
+-----------+----------+-------+---------------+
| StudentID | Course   | Grade | Department    |
+-----------+----------+-------+---------------+
|       201 | Cloud    | B     | Cloud Tech    |
|       201 | WebDev   | A     | Software Eng  |
|       202 | CyberSec | C     | Cybersecurity |
|       202 | DataViz  | A     | Software Eng  |
|       202 | IoT      | B     | Cloud Tech    |
|       203 | DevOps   | A     | Software Eng  |
|       204 | Cloud    | B     | Cloud Tech    |
|       204 | DataViz  | A     | Software Eng  |
|       205 | CyberSec | A     | Cybersecurity |
|       205 | IoT      | C     | Cloud Tech    |
|       205 | WebDev   | B     | Software Eng  |
+-----------+----------+-------+---------------+
11 rows in set (0.00 sec)
```

```
mysql> select * from  Course_3NF;
+----------+---------------+
| Course   | Department    |
+----------+---------------+
| Cloud    | Cloud Tech    |
| CyberSec | Cybersecurity |
| DataViz  | Software Eng  |
| DevOps   | Software Eng  |
| IoT      | Cloud Tech    |
| WebDev   | Software Eng  |
+----------+---------------+
6 rows in set (0.01 sec)
```

```
mysql> select * from Enrollment_3NF;
+-----------+----------+-------+
| StudentID | Course   | Grade |
+-----------+----------+-------+
|       201 | Cloud    | B     |
|       201 | WebDev   | A     |
|       202 | CyberSec | C     |
|       202 | DataViz  | A     |
|       202 | IoT      | B     |
|       203 | DevOps   | A     |
|       204 | Cloud    | B     |
|       204 | DataViz  | A     |
|       205 | CyberSec | A     |
|       205 | IoT      | C     |
|       205 | WebDev   | B     |
+-----------+----------+-------+
11 rows in set (0.01 sec)
```

- **4th Normal Form (4NF):**

4NF addresses multivalued dependencies (MVDs). A relation is in 4NF if it is in Boyce-Codd Normal Form (BCNF) and does not contain any multivalued dependencies.

What is a Multivalued Dependency?

A multivalued dependency (MVD) occurs when one attribute in a table determines a set of values for another attribute, and the determined set of values is independent of other attributes in the table. This creates redundancy and can lead to data anomalies.

```
mysql> select * from  Enrollment_BCNF2;
+-----------+-----------+---------------+
| StudentID | Course    | Hobby         |
+-----------+-----------+---------------+
|       201 | Cloud     | Photography   |
|       201 | WebDev    | Gaming        |
|       202 | CyberSec  | Hiking        |
|       202 | DataViz   | Gaming        |
|       202 | IoT       | Gaming        |
|       203 | DevOps    | Blogging      |
|       204 | Cloud     | Photography   |
|       204 | DataViz   | Gaming        |
|       205 | CyberSec  | Gaming        |
|       205 | IoT       | Traveling     |
|       205 | WebDev    | Blogging      |
+-----------+-----------+---------------+
11 rows in set (0.01 sec)

mysql> select * from  StudentCourses_4NF;
+-----------+-----------+
| StudentID | Course    |
+-----------+-----------+
|       201 | Cloud     |
|       204 | Cloud     |
|       202 | CyberSec  |
|       202 | DataViz   |
|       203 | DevOps    |
|       201 | WebDev    |
+-----------+-----------+
6 rows in set (0.02 sec)

mysql> select * from StudentHobbies_4NF;
+-----------+---------------+
| StudentID | Hobby         |
+-----------+---------------+
|       201 | Gaming        |
|       201 | Photography   |
|       202 | Gaming        |
|       202 | Hiking        |
|       203 | Blogging      |
+-----------+---------------+
```

- **5th Normal Form (5NF):**

5NF, also called Project-Join Normal Form (PJNF), deals with join dependencies and ensures that a relation cannot be decomposed into smaller relations without losing information. A table is in 5NF if it is in 4NF and if every join dependency is a trivial join dependency.

What is a Join Dependency?

A join dependency occurs when a relation can be decomposed into multiple smaller relations, but these relations can only be reconstructed by performing a join operation. If a table can be decomposed into smaller tables without loss of information, it's violating 5NF.

Before 5NF:

```
mysql> select * from  ConsultantProjectSkill;
+------------+---------+--------+
| Consultant | Project | Skill  |
+------------+---------+--------+
| Liam       | Alpha   | Cloud  |
| Liam       | Alpha   | DevOps |
| Liam       | Beta    | Cloud  |
| Liam       | Beta    | DevOps |
| Zara       | Alpha   | Cloud  |
| Zara       | Alpha   | DevOps |
| Zara       | Beta    | Cloud  |
| Zara       | Beta    | DevOps |
+------------+---------+--------+
8 rows in set (0.01 sec)
```

After 5NF:

```
mysql>  select * from ConsultantProject_5NF;
+------------+---------+
| Consultant | Project |
+------------+---------+
| Liam       | Alpha   |
| Liam       | Beta    |
| Zara       | Alpha   |
| Zara       | Beta    |
+------------+---------+
4 rows in set (0.00 sec)

mysql> select * from  ConsultantSkill_5NF;
+------------+--------+
| Consultant | Skill  |
+------------+--------+
| Liam       | Cloud  |
| Liam       | DevOps |
| Zara       | Cloud  |
| Zara       | DevOps |
+------------+--------+
4 rows in set (0.00 sec)

mysql> select * from ProjectSkill_5NF;
+---------+--------+
| Project | Skill  |
+---------+--------+
| Alpha   | Cloud  |
| Alpha   | DevOps |
| Beta    | Cloud  |
| Beta    | DevOps |
+---------+--------+
4 rows in set (0.00 sec)
```

CHAPTER 5 CONCURRENCY CONTROL AND RECOVERY MECHANISAM

5.1 Concurrency Control in Organ Donation System

Concurrency control ensures that multiple users can safely access or modify the database simultaneously without causing inconsistencies or conflicts. In the Organ Donation System,

concurrency control is vital when multiple doctors or hospitals try to reserve or update the status of the same organ.

a. Lock-Based Concurrency Using Transactions

We use transactions and row-level locks to prevent race conditions. The SELECT ... FOR UPDATE query locks the row until the transaction completes. Example: Reserving an Organ (Flask + SQLAlchemy)

Set appropriate isolation level (e.g., REPEATABLE READ or SERIALIZABLE) for data consistency.

```
from flask_sqlalchemy import SQLAlchemy
from sqlalchemy.exc import SQLAlchemyError
db = SQLAlchemy() def
reserve_organ(organ_id, patient_id): try:
    # Begin transaction with row-level locking
organ = db.session.execute(
        "SELECT * FROM organ_available WHERE organ_id = :id FOR UPDATE",
        {'id': organ_id} ).fetchone() if
organ and organ.status == 'Available': #
Update organ status to 'Reserved'
db.session.execute(
            "UPDATE organ_available SET status = 'Reserved' WHERE organ_id = :id",
            {'id': organ_id}
        )
        # Record the transaction
db.session.execute(
            "INSERT INTO transaction (organ_id, patient_id, status) VALUES (:oid, :pid,
'Pending')",
            {'oid': organ_id, 'pid': patient_id}
        )
        # Commit transaction
db.session.commit() return "Organ
reserved successfully." else:
        # Rollback if not available
db.session.rollback() return "Organ not
available."

  except SQLAlchemyError as e:
    # Rollback on error
db.session.rollback()
    return f"Transaction failed. Rolled back. Error: {str(e)}"
```

5.2 Recovery Mechanisms

Recovery mechanisms are used to bring the database back to a consistent state after a crash or system failure.

a. Write-Ahead Logging (WAL)

MySQL's InnoDB engine uses WAL by default. Enable binary logging in your MySQL configuration file:

```
[mysqld]
# Enable binary logging log_bin
= mysql-bin
# Use row-based logging for better recovery of transactional changes binlog_format
= ROW
# Force InnoDB to flush logs to disk after every transaction
innodb_flush_log_at_trx_commit = 1 # Ensure full crash-
safe recovery sync_binlog = 1
sudo systemctl restart mysql  mysqldump -u root -p organ_donation_db >
/backups/organ_donation_backup.sql
mysql -u root -p organ_donation_db < /backups/organ_donation_backup.sql
```

CHAPTER 6 CODE FOR THE PROJECT

```
from flask import Flask,render_template,session,request,redirect,url_for,flash import
mysql.connector import hashlib import matplotlib.pyplot as plt import numpy as np
import os mydb= mysql.connector.connect(
host='localhost',
user='root', password='punith@2005',
database  ='DBMS_PROJECT',
port='3306'
)
import mysql.connector
# mydb = mysql.connector.connect(
# host=os.getenv("DB_HOST", "localhost"),
# user=os.getenv("DB_USER", "root"),
# password=os.getenv("DB_PASSWORD", "punith@2005"),
# database=os.getenv("DB_NAME", "DBMS_PROJECT"),
# port=int(os.getenv("DB_PORT", 3306))
# ) mycursor = mydb.cursor(buffered=True) app
= Flask(_name_)
@app.route("/",methods  =  ['POST',
    'GET']) @app.route("/home",methods = ['POST','GET'])
def home(): if not
session.get('login'): return render_template('login.html'),401
else:  if session.get('isAdmin') :
```

```
returnrender_template('home.html',username=session.get('username'))
else :
return home_student()
@app.route("/login",methods = ['GET','POST']) def login(): if
request.method=='POST' :
query = """SELECT * FROM login WHERE username='%s'"""
%(request.form['username'])
mycursor.execute(query) res =
mycursor.fetchall()if mycursor.rowcount == 0:
return home() if
request.form['password'] != res[0][1]:
return render_template('login.html') else:
session['login'] = True session['username'] =
request.form['username']
session['password'] = request.form['password'] session['isAdmin'] =
(request.form['username']=='admin')return home() return
render_template('login.html')
@app.route("/show_update_detail",methods=['POST','GET'])def show_update_detail():
if not session.get('login'):
return redirect( url_for('home') ) if request.method=='POST': if request.form['User_ID'] ==": return
render_template("search_detail.html") qry = "Select * from User where User.User_ID = %s"
%(request.form['User_ID']) qry1 = "Select * from User_phone_no where User_ID = %s"
%(request.form['User_ID']) mycursor.execute(qry) not_found=False res=() if(mycursor.rowcount
> 0): res =
mycursor.fetchone() else: not_found=True fields = mycursor.column_names qry_upd =
"Select * from User where User_ID = %s" %(request.form['User_ID'])
mycursor.execute(qry_upd) upd_res = () if(mycursor.rowcount
> 0): upd_res = mycursor.fetchone() fields_upd = mycursor.column_names mycursor.execute(qry1)
phone_no = mycursor.fetchall() qry_pat = "select Patient_ID, organ_req, reason_of_procurement,
Doctor_name from Patient inner join Doctor on Doctor.Doctor_ID = Patient.Doctor_ID and
User_ID = %s"
%(request.form['User_ID'])qry_don="selectDonor_ID, organ_donated, reason_of_donation,
Organization_name from Donor inner join Organization on
Organization.Organization_ID =
Donor.Organization_ID and User_ID = %s" %(request.form['User_ID']) qry_trans = "select distinct
Transaction.Patient_ID, Transaction.Donor_ID, Organ_ID, Date_of_transaction, Status from
Transaction, Patient, Donor where (Patient.User_ID = %s and Patient.Patient_ID =
Transaction.Patient_ID) or (Donor.User_Id=
%s and
Donor.Donor_ID=Transaction.Donor_ID)"
%((request.form['User_ID']),(request.form['User_ID']))
# res_pat = () res_dnr = () res_trans = () mycursor.execute(qry_pat)
if(mycursor.rowcount >
0):
```

```
res_pat = mycursor.fetchall()
fields_pat = mycursor.column_names
# mycursor.execute(qry_don)
if(mycursor.rowcount > 0): res_dnr =
mycursor.fetchall() fields_dnr =
mycursor.column_names
#
mycursor.execute(qry_trans) if(mycursor.rowcount
> 0):
 res_trans= mycursor.fetchall() fields_trans
= mycursor.column_names
print(res_trans) if("show" in request.form):
return render_template('show_detail_2.html',res = res,fields = fields, not_found=not_found,
phone_no = phone_no, res_dnr = res_dnr, res_pat = res_pat,res_trans = res_trans,fields_trans =
fields_trans, fields_dnr = fields_dnr, fields_pat = fields_pat) if("update" in request.form):
returnrender_template('update_detail.html',res = upd_res,fields = fields_upd,
not_found=not_found) if "delete" in request.form: if not_found:
 returnrender_template('show_detail_2.html',res = res,fields = fields,
not_found=not_found, phone_no = phone_no, res_dnr= res_dnr, res_pat =
res_pat,res_trans
= res_trans,fields_trans = fields_trans, fields_dnr = fields_dnr, fields_pat = fields_pat) else: qry2
= "DELETE FROM User where User_ID = %s" %(request.form['User_ID'])
mycursor.execute(qry2) mydb.commit() return
    render_template("home.html")
@app.route("/search_detail",methods = ['POST','GET']) def
search_detail(): if not session.get('login'): return
redirect( url_for('home') ) return
render_template('search_detail.html')
#--------------Adding Information--------------------------
@app.route("/add_<id>_page",methods = ['POST','GET']) def
add_page(id): if not session.get('login'): return redirect(
url_for('home') ) qry = "SELECT * from "
    + id.capitalize() mycursor.execute(qry) fields =
mycursor.column_namesreturn
render_template('add_page.html',success=request.args.get('success'),
error=request.args.get('error'), fields = fields, id= id)
@app.route("/add_User", methods=['POST','GET']) def
add_User(): if not session.get('login'): return redirect(
url_for('home') ) qry = "SELECT * from User"
mycursor.execute(qry) fields = mycursor.column_names
val = () for field in fields:
 temp =request.form.get(field) if field not in ['User_ID','Medical_insurance']
and temp != '':
```

```
 temp ="\'"+temp+"\'" if
temp == '':
 temp = 'NULL'
val = val + (temp,) qry = "INSERT INTO User Values
(%s,%s,%s,%s,%s,%s,%s,%s)"%val print(qry)    success = True
error = False try: mycursor.execute(qry) except: print("Error : User
not Inserted") error = True
 success = False mydb.commit() return redirect(url_for('add_page', id='User',
error=error,success=success))
@app.route("/add_User_phone_no", methods=['POST','GET']) def add_User_phone_no(): if
not session.get('login'):
 return redirect( url_for('home')
    ) qry = "SELECT * from
User_phone_no"  mycursor.execute(qry)
    fields = mycursor.column_names val
= () for field in fields: temp = request.form.get(field) if field not in
['User_ID','Phone_no'] and temp != '':  temp ="\'"+temp+"\'" if temp == '':
temp = 'NULL' val
= val + (temp,)
qry = "INSERT INTO User_phone_no Values (%s,%s)"%val
print(qry)    success = True error = False try: mycursor.execute(qry)
except:
 print("Error : User not Inserted") error = True success = False
mydb.commit() return redirect(url_for('add_page', id='User_phone_no',
    error=error,success=success)) @app.route("/add_Patient",
    methods=['POST','GET'])    def add_Patient():
if not session.get('login'): return redirect(
url_for('home') ) qry =
    "SELECT * from Patient"
mycursor.execute(qry)    fields = mycursor.column_names
val = ()
@app.route('/contact_admin',methods=['GET','POST']) def
contact_admin():    if not session.get('login') or
session.get('isAdmin'):
return redirect( url_for('home') ) username = session.get('username') message =
request.form['message'] qry = "insert into Messages (username,message) values
(\'"+username+"\',\'"+message+"\')" success = True error = False try:
mycursor.execute(qry)
except:    print("Error")
error = True success =
False mydb.commit()

return render_template('contact_admin_page.html',error=error,success=success)
```

```
@app.route('/see_messages',methods=['GET','POST']) def
see_messages(): if not session.get('login') or not
session.get('isAdmin'): return redirect( url_for('home') )
qry = "Select * from Messages" mycursor.execute(qry)
msg =
mycursor.fetchall() return
render_template('see_messages.html',msg=msg)
@app.route('/seen_message',methods=['GET','POST']) def
seen_message(): if not session.get('login') or not
session.get('isAdmin'):
return redirect( url_for('home') ) print(request.form['id']) msg_id = request.form['id']
qry = "delete from Messages where
message_id=\'"+msg_id+"\'" mycursor.execute(qry) mydb.commit()
return redirect(url_for('see_messages'))
@app.route('/statistics', methods=['GET','POST']) def stats():
if not session.get('login') or not session.get('isAdmin'): return redirect(
url_for('home') )qry = "select organ_donated, count(Donor_ID)
from Donor group by organ_donated" mycursor.execute(qry) stats_donor
= mycursor.fetchall()
A = []
B = []for organ in stats_donor:
A.append(organ[0])
B.append(organ[1]) plt.pie(B, labels =
A)
plt.savefig('./static/donor_stat.png')
# plt.show() plt.close() A.clear() B.clear() qry = "select organ_req,
count(Patient_Id) from Patient group by organ_req" mycursor.execute(qry)
stats_patient = mycursor.fetchall()
A = [] B = [] for
Patient in stats_patient:
A.append(Patient[0])
B.append(Patient[1]) plt.pie(B, labels
= A)
plt.savefig('./static/Patient_stat.jpeg')
# plt.show()
plt.close()
qry = "select distinct Organ_donated from Transaction inner join Donor on
Transaction.Donor_ID = Donor.Donor_ID"
mycursor.execute(qry) list =
mycursor.fetchall() organ_list = [] for
organ in list: print(organ)
```

```
organ_list.append(organ[0])
print(organ)   A.clear()   B.clear()   for
organ in organ_list:
qry = "select count(*) from Transaction inner join Donor on Donor.Donor_ID =
Transaction.Donor_ID where Organ_donated = '%s' and Status = 1" %organ print(qry)
mycursor.execute(qry) a = mycursor.fetchone() A.append(a[0])
qry = "select count(*) from Transaction inner join Donor on Donor.Donor_ID =
Transaction.Donor_ID where Organ_donated = '%s' and Status = 0" %organ print(qry)
mycursor.execute(qry) b = mycursor.fetchone() B.append(b[0])   print(A)          print(B)
        print(organ_list) N = len(organ_list) fig, ax =
 plt.subplots() ind = np.arange(N) width =  0.05 plt.bar(ind,
A, width, label='SUCCESS') plt.bar(ind + width, B,
width,label='FAILUR E' plt.ylabel('Number
of transplantation')
plt.xlabel('Organ')
plt.title('SUCCESS V/S FAILURE IN ORGAN TRANSPLANTATION') plt.xticks(ind
      +        width   /        2,        organ_list)      plt.legend(loc='best')
plt.savefig('./static/success.jpeg')
return render_template('statistics.html') if
_name_ == "_main_": app.secret_key
=     'sec     key'     app.config['SESSION_TYPE']          'filesystem'      app.run(debug=True)
# app.run(debug=True, port=3306)
```

CHAPTER 7

RESULTS

Screenshots

1) Login Page

2) Main Page – GUI

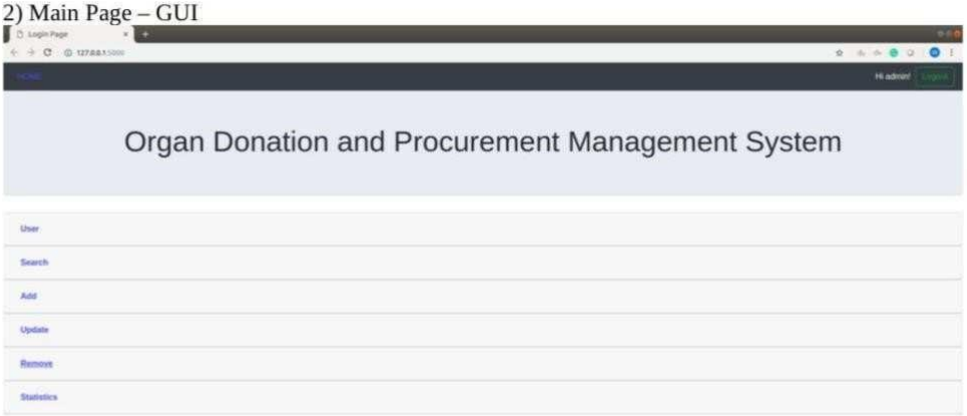


Fig 7.1: user and password (login page)

4) Searching Option

User_ID	Name	Date_of_Birth	Medical_insurance	Medical_history	Street	City	State
3	Name-3	1976-06-04	0	NIL	Street-3	Mumbai	Maharashtra
8	Name-8	1973-04-12	1	NIL	Street-8	Mumbai	Maharashtra
9	Name-9	1976-11-01	0	NIL	Street-9	Mumbai	Maharashtra
11	Name-11	1975-01-06	1	NIL	Street-11	Mumbai	Maharashtra
12	Name-12	1983-11-01	1	NIL	Street-12	Mumbai	Maharashtra
14	Name-14	1975-10-12	1	NIL	Street-14	Mumbai	Maharashtra
17	Name-17	1974-03-19	0	NIL	Street-17	Mumbai	Maharashtra
21	Name-21	1975-10-12	1	NIL	Street-21	Mumbai	Maharashtra
23	Name-23	1987-03-30	1	NIL	Street-23	Mumbai	Maharashtra
29	Name-29	1985-01-04	0	NIL	Street-29	Mumbai	Maharashtra
40	Name-40	1978-02-20	1	NIL	Street-40	Mumbai	Maharashtra
41	Name-41	1987-02-19	0	NIL	Street-41	Mumbai	Maharashtra
42	Name-42	1975-11-30	0	NIL	Street-42	Mumbai	Maharashtra
43	Name-43	1986-12-18	1	NIL	Street-43	Mumbai	Maharashtra

Fig 7.2 :Search operation

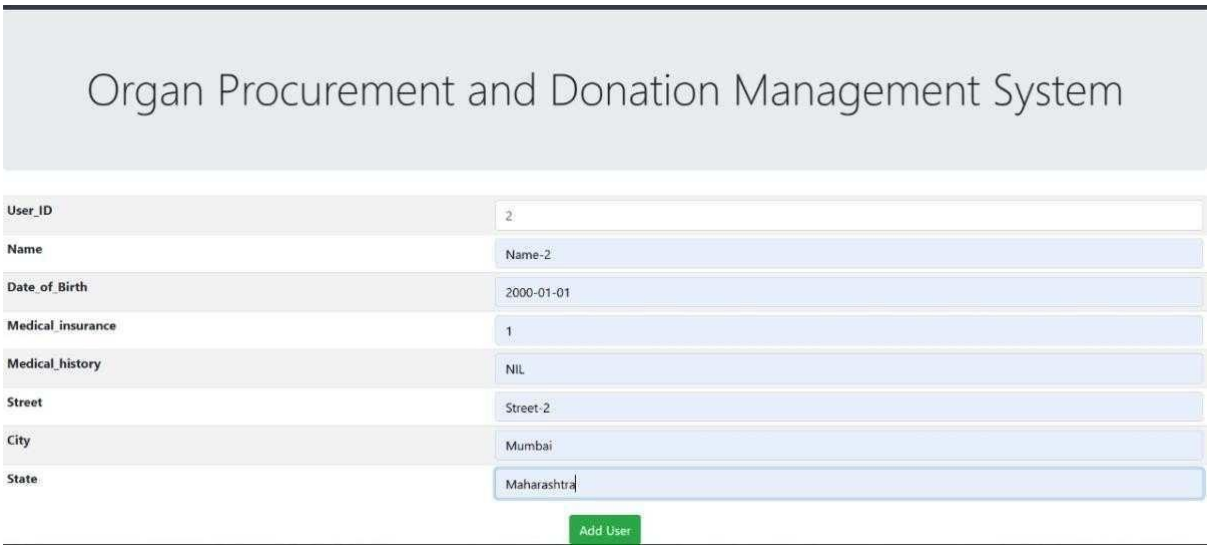


Fig 7.3 : Add operation

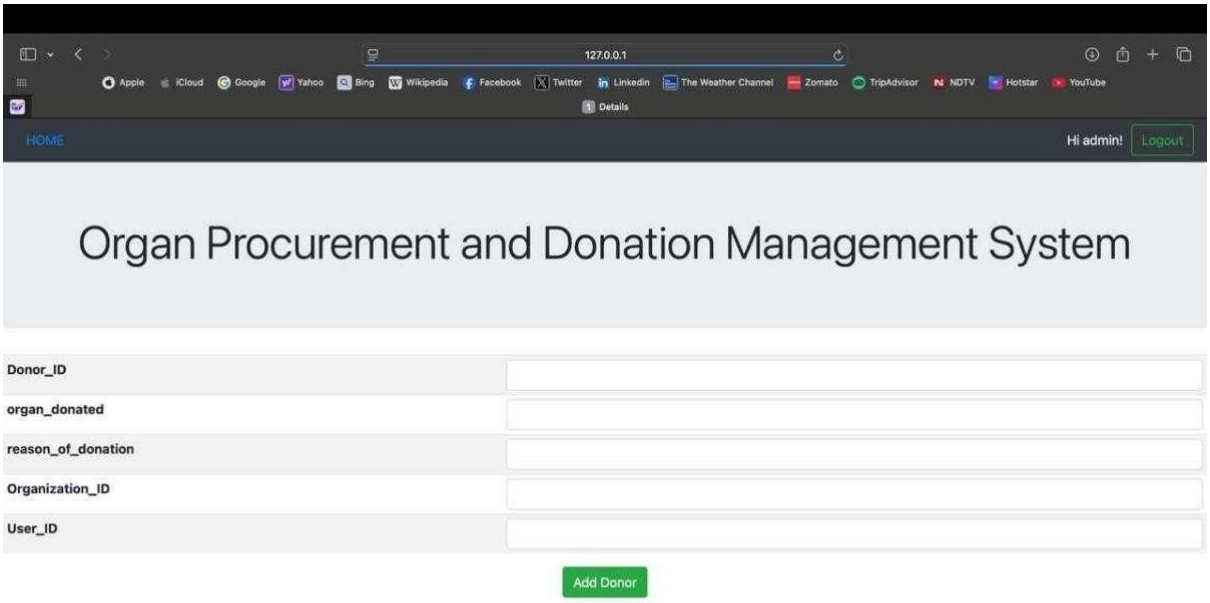


Fig 7. 4 :update details operation

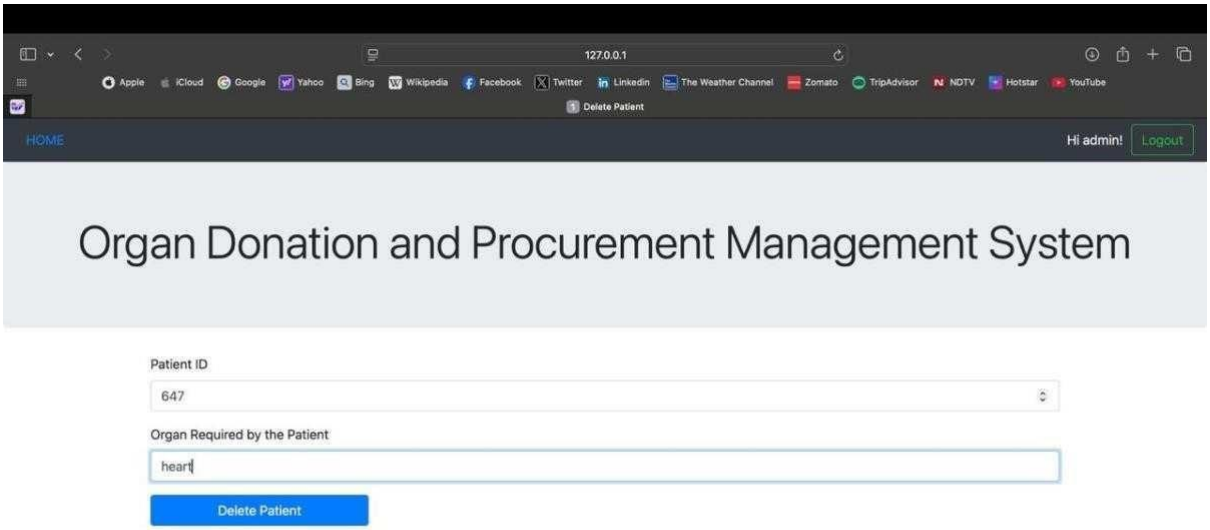


fig 7.5 :Delete operation

REFERENCE

1. Ministry of Health and Family Welfare, Government of India – *National Organ and Tissue Transplant Organization (NOTTO)*. https://notto.mohfw.gov.in
2. World Health Organization (WHO) – *Organ Transplantation*. https://www.who.int/news-room/fact-sheets/detail/transplantation-of-human-cellstissuesand-organs
3. MySQL Documentation – *MySQL 8.0 Reference Manual*. https://dev.mysql.com/doc/
4. CodeWithHarry – *MySQL Database Tutorial for Beginners*. https://www.youtube.com/c/CodeWithHarry
5. GeeksforGeeks – *Organ Transplant Management System Project*. https://www.geeksforgeeks.org